OPENSHORTS — COMPLETE ARCHITECTURAL AUDIT REPORT & TECHNICAL REPAIR PLAN

Project: FluxClip Studio / OpenShorts Lightweight Frontend-Driven Pipeline | **Status:** Pre-Implementation Technical Audit

PART 1: INDEPENDENT VERIFICATION OF THE 7 REPORTED CONFLICTS

ISSUE 1: Master Audio Binary Is Never Uploaded

1. **Is the issue actually present?** YES (Confirmed 100%)
2. **Exact Root Cause:**
   - In `src/components/UploadsScreen.tsx` (lines 109–123), `handleAudioFileChange` stores `rawFile: picked` inside the React state object `MasterAudioPayload`, and `useOpenShortsViewModel.ts` holds `masterAudio` in state.
   - However, in `src/viewmodels/useOpenShortsViewModel.ts` (lines 189–218), `dispatchToOpenShortsBackend()` only calls `uploadFileInChunks(localFile.rawFile)` when `sourceValidation.sourceType === "file"`. It never calls `uploadFileInChunks(masterAudio.rawFile)`.
   - Instead, lines 208–215 serialize only `{ fileName, format, sizeBytes, durationMs }` into the JSON body of `POST /api/process`.
   - On the backend (`server.ts`, lines 627–634 & 412–416), `job.masterAudio` contains only metadata. When `buildTargetedFfmpegExtractCommands` constructs `-i "${masterAudioName}"`, no audio file exists in `STAGING_ROOT`.
3. **Is the proposed fix technically correct?** Yes. Uploading the master audio binary via the chunked upload infrastructure (`audioUploadId`), verifying its existence on disk in `STAGING_ROOT`, and running `ffprobe` on the staged audio file to verify format and true duration is the right design.
4. **Could that fix create any new problems or conflicts?**
   - **Conflict A (Cleanup Timing):** In `server.ts` (line 686), `uploadSessions.delete(uploadId)` deletes the session immediately upon job creation, and `cleanupTempPaths(job.tempFiles)` deletes staged files as soon as the job finishes or fails. If a duplicate/retried request references the same `audioUploadId`, it would fail with 404 unless upload sessions are finalized cleanly and bound to a job-specific directory.
   - **Conflict B (Sequential Upload Latency):** Uploading both a large local video and a master WAV/MP3 sequentially without granular progress feedback can make the UI appear frozen during `"Uploading chunks..."`.
5. **Safest Recommended Solution:**
   - Extend the chunked upload flow to upload `masterAudio.rawFile` and produce a dedicated `audioUploadId` alongside `videoUploadId` (when local video is used).
   - Add a `POST /api/upload/complete` finalization step that verifies `receivedBytes === totalSizeBytes` and runs `/usr/bin/ffprobe` on the staged file to extract authoritative container format and `durationMs` before `/api/process` is called.
   - Reject `/api/process` with HTTP 422 if `audioUploadId` does not resolve to a verified, non-empty audio file on disk.

ISSUE 2: Chunked Upload Only Transmits 512 KB

1. **Is the issue actually present?** YES (Confirmed 100%)

2. **Exact Root Cause:**
    - In `src/viewmodels/useOpenShortsViewModel.ts` (lines 34–67), `uploadFileInChunks(file: File)` initializes an upload session via `/api/upload/init`, then slices only `file.slice(0, Math.min(file.size, 512 * 1024))`, sends one `POST /api/upload/chunk` request, and immediately returns `uploadId`.
    - Furthermore, `server.ts` (lines 682–687) never checks whether `session.receivedBytes === session.totalSizeBytes` before accepting `uploadId` in `POST /api/process`.
3. **Is the proposed fix technically correct?** Yes. Looping through all chunks sequentially with byte-offset tracking, retry logic, and server-side completeness verification is required.
4. **Could that fix create any new problems or conflicts?**
    - **Conflict A (JSON Base64 Overhead & Call Stack):** `express.json({ limit: "16mb" })` is active in `server.ts` (line 477), while `MAX_CHUNK_SIZE_BYTES` is 8MB. Base64 encoding inflates 8MB of binary data to ~10.67MB of JSON string, and byte-by-byte `String.fromCharCode` in browser JS blocks the main thread on large files.
    - **Conflict B (Out-of-Order / Retry Duplication):** Currently `server.ts` (line 562) blindly runs `fs.appendFileSync(session.tempFilePath, buffer)`. If a network retry resends chunk `index = 2`, `appendFileSync` will append duplicate bytes and corrupt the MP4/MP3 container.
5. **Safest Recommended Solution:**
    - Use a configurable chunk size (default 2 MB raw binary = ~2.67 MB base64) and convert chunks asynchronously.
    - Send explicit `chunkIndex` and `byteOffset` in `POST /api/upload/chunk`.
    - On the server, enforce idempotent offset writing: verify `byteOffset === session.receivedBytes` (or write at exact position via file descriptor `fs.writeSync`) so retries never duplicate data.
    - Require `session.receivedBytes === session.totalSizeBytes` before a session can be consumed by `/api/process`.

ISSUE 3: FFmpeg Commands Are Generated But Never Executed

1. **Is the issue actually present?** YES (Confirmed 100%)
2. **Exact Root Cause:**
    - **Environment Check:** `/usr/bin/ffmpeg` and `/usr/bin/ffprobe` are installed and available in the container environment, but `server.ts` never invokes them.
    - In `server.ts` (lines 156–192 & 248–261), `buildTargetedFfmpegExtractCommands()` and `buildFfmpegBurnInRenderCommand()` only return formatted strings (`ffmpegExtractCmd` and `ffmpegBurnInCmd`).
    - In `executeLightweightPipeline()` (lines 412–450), no child process is ever started; no `.mp4`, `.mp3`, or `.ass` files are written to disk.
    - In `GET /api/artifacts/:jobId/:clipFile` (lines 751–798), the endpoint returns `res.json({ jobId, clipId, ... })` instead of streaming an MP4 file.
3. **Is the proposed fix technically correct?** Yes. Spawning `/usr/bin/ffmpeg` with an explicit argument array (`spawn("/usr/bin/ffmpeg", args)`) and serving the resulting MP4 file from `GET /api/artifacts/:jobId/:clipFile` with `Content-Type: video/mp4` is essential.
4. **Could that fix create any new problems or conflicts?**
    - **Conflict A (Premature Cleanup Bug in `server.ts` line 458):** At the end of `executeLightweightPipeline()`, `cleanupTempPaths(job.tempFiles)` runs immediately. If rendered clip MP4s are stored in `job.tempFiles`, they would be deleted before the user can preview or download them on Screen 2.

- **Conflict B (HTML5 Video Seeking Requires HTTP Range Support):** Returning a stream without `Accept-Ranges: bytes` and `206 Partial Content` breaks video scrubbing in Chrome and Safari. Express `res.sendFile(absoluteClipPath)` natively handles HTTP Range headers.
- **Conflict C (Stream Mapping & Faststart):** Without explicit `-map 0:v:0 -map 1:a:0 -pix_fmt yuv420p -movflags +faststart`, FFmpeg may fail to mux external audio or produce non-streamable MP4s.

5. **Safest Recommended Solution:**
   - Create two distinct per-job directories under `STAGING_ROOT`: `STAGING_ROOT/jobs/<jobId>/input/` (deleted immediately when rendering finishes) and `STAGING_ROOT/jobs/<jobId>/output/` (retained until 30-minute TTL expiration).
   - Execute `/usr/bin/ffmpeg` via `child_process.spawn` with array arguments, capturing `stderr` and enforcing per-stage timeouts.
   - Generate a real `.ass` subtitle file on disk with slice-relative timestamps and burn it in with `-vf crop=ih*9/16:ih,scale=720:1280,subtitles=... -map 0:v:0 -map 1:a:0 -c:v libx264 -preset veryfast -pix_fmt yuv420p -c:a aac -b:a 128k -movflags +faststart -shortest`.
   - Serve `/api/artifacts/:jobId/:clipFile` using `res.sendFile(outputFilePath)` with `Content-Type: video/mp4`.

ISSUE 4: Clip Preview Plays the Original Video From 00:00

1. **Is the issue actually present?** YES (Confirmed 100%)
2. **Exact Root Cause:**
   - In `src/viewmodels/useOpenShortsViewModel.ts` (lines 183–186 & 251–254), `localObjectUrl = URL.createObjectURL(localFile.rawFile)` is passed into `mapOpenShortsClipsToArtifacts(statusData.result.clips, localObjectUrl)`.
   - In `src/utils/pipelineLogic.ts` (lines 613–614), `mapOpenShortsClipsToArtifacts` sets `videoUrl: localPreviewUrl`.
   - Consequently, for Local Uploads, every clip receives the uncut blob URL of the full source video starting at `0:00`; for Direct URLs and YouTube URLs, `localPreviewUrl` is `undefined` so no video plays at all; and `URL.createObjectURL` is never revoked.
3. **Is the proposed fix technically correct?** Yes. Map `videoUrl` directly to `apiClip.video_url` (the backend signed MP4 endpoint) for all 3 input modes.
4. **Could that fix create any new problems or conflicts?** On mobile browsers, `<video preload="metadata">` in carousel thumbnails can show a black frame at `0.0s` unless `#t=0.1` or a poster thumbnail is used.
5. **Safest Recommended Solution:** Remove `localPreviewUrl` from `mapOpenShortsClipsToArtifacts`, bind `videoUrl: apiClip.video_url`, and use `#t=0.1` for compact carousel thumbnails.

ISSUE 5: Remote Video Duration Is Not Validated

1. **Is the issue actually present?** YES (Confirmed 100%)
2. **Exact Root Cause:**
   - In `src/viewmodels/useOpenShortsViewModel.ts` (lines 163–164), `effectiveVideoDurationMs` falls back to `masterAudio?.durationMs || 0` when `videoDurationMs` is 0 (which is always true for remote URLs). This compares `masterAudio.durationMs` against itself, bypassing A/V duration sync validation.
   - In `server.ts` (lines 619–622), a hardcoded `1860000` ms fallback still exists.

3. **Is the proposed fix technically correct?** Yes. Probe both the video source and the staged master audio file using `/usr/bin/ffprobe` and compare their actual durations against the 2,500 ms tolerance.
4. **Could that fix create any new problems or conflicts?** If the frontend rejects `videoDurationMs === 0` before calling the backend on a remote URL, remote URLs could not be submitted.
5. **Safest Recommended Solution:** Remove the `masterAudio.durationMs` fallback and `1860000` constant. Validate local video duration on the client, and perform authoritative `/usr/bin/ffprobe` duration extraction and 2.5s sync validation for both local and remote sources on the backend in `/api/process`.

ISSUE 6: Frontend Job Polling Times Out Too Early

1. **Is the issue actually present?** YES (Confirmed 100%)
2. **Exact Root Cause:**
   - In `src/viewmodels/useOpenShortsViewModel.ts` (lines 231–232), the loop runs `20 attempts * 180ms = 3.6 seconds`.
   - If the job takes longer than 3.6 seconds, execution exits the loop without resetting `setIsGenerating(false)` or `setPipelineStage(null)`, leaving the UI button permanently stuck.
3. **Is the proposed fix technically correct?** Yes. Expose granular backend job states (`queued`, `validating`, `analyzing`, `extracting`, `transcribing`, `rendering`, `completed`, `failed`, `cancelled`) and poll until a terminal state with an `AbortController` and a `finally` block.
4. **Could that fix create any new problems or conflicts?** Unmounting during active polling could trigger state updates on an unmounted component unless aborted via `useEffect` cleanup.
5. **Safest Recommended Solution:** Poll every 1000ms using an `AbortController` ref cleaned up on unmount, tolerate up to 3 transient network retries, and guarantee `setIsGenerating(false)` in a `finally` block.

ISSUE 7: Android UI Has No Transcript or Master Audio Pickers

1. **Is the issue actually present?** YES (Confirmed 100%)
2. **Exact Root Cause:**
   - `OpenShortsViewModel.kt` requires `masterAudio != null` and `transcriptText.isNotBlank()`, but `Screen1UploadsTab` in `FluxClipScreen.kt` only renders the URL field and local video picker. It also does not render `uiState.validationError`.
3. **Is the proposed fix technically correct?** Yes. Add SAF document pickers for `.srt/.txt` and `.mp3/.wav`, extract display name, size, and duration via `ContentResolver` and `MediaMetadataRetriever`, and wire remove/replace controls and validation error display.
4. **Could that fix create any new problems or conflicts?** Using `uri.lastPathSegment` on Android SAF URIs returns opaque IDs like `document:1000000042` instead of the real filename.
5. **Safest Recommended Solution:** Query `OpenableColumns.DISPLAY_NAME` and `OpenableColumns.SIZE` via `ContentResolver` and extract duration with `MediaMetadataRetriever`.

PART 2: ADDITIONAL ARCHITECTURAL CONFLICTS DISCOVERED

- **Conflict 8 (Synthetic `yt-dlp://` Protocol URI):** `buildUnifiedMediaSource` in `pipelineLogic.ts` outputs `yt-dlp://targeted-range-stream?url=...`, which `/usr/bin/ffmpeg` rejects as an unknown protocol, and `yt-dlp` is not installed in `/usr/bin`. We must resolve real HTTPS stream URLs or return a clear, structured error when YouTube extraction is restricted/unavailable.
- **Conflict 9 (SSRF Risk on Direct URLs):** `classifyRemoteVideoUrl` and `probeDirectUrlRangeSupport` allow arbitrary internal/loopback/metadata hosts. We must resolve DNS via `dns.promises.lookup` and block loopback, link-local, and RFC1918 private IPs before invoking `fetch` or `ffprobe` / `ffmpeg`.

- **Conflict 10 (Faster-Whisper Relative vs. Absolute Subtitle Timestamps):** Sliced MP4 clips start at `00:00:00.000`, so burned-in `.ass` subtitles must use slice-relative timestamps (`0 .. clipDurationMs`), while API metadata exposes both relative and source-timeline timestamps.
- **Conflict 11 (Deduplication Hash Collision):** `dedupeKey` hashes `transcriptText.length` instead of the full transcript content and upload IDs.
- **Conflict 12 (Premature Deletion of Job Output Files):** `cleanupTempPaths(job.tempFiles)` runs immediately when `executeLightweightPipeline` finishes; input staging files must be separated from output clip MP4s so output clips remain available for preview and download.

PART 3: EXACT FILES REQUIRING MODIFICATIONS

Layer	File Path	Planned Modifications
Shared Logic	`/src/utils/pipelineLogic.ts`	Add expanded `JobStage` union, fix `mapOpenShortsClipsToArtifacts` to bind `videoUrl: apiClip.video_url`, remove synthetic `yt-dlp://` URI scheme.
Unit Tests	`/src/utils/pipelineLogic.test.ts`	Add unit tests for per-clip artifact URL mapping, A/V sync mismatch rejection, and transcript bounds validation.
Backend Engine	`/server.ts`	Offset-verified chunked uploads + `/api/upload/complete` with `ffprobe`, SSRF protection, real `spawn("/usr/bin/ffmpeg", ...)` video/audio slicing and `.ass` burn-in, separate input/output temp directories, binary `video/mp4` streaming via `res.sendFile`.
Frontend ViewModel	`/src/viewmodels/useOpenShortsViewModel.ts`	Full multi-chunk upload loop for both video and master audio, remove remote duration fallback, `AbortController` stage polling loop.
Frontend UI	`/src/components/UploadsScreen.tsx` `/src/components/PodcasterFrame.tsx`	Show upload progress and clear/replace buttons; bind distinct clip artifact URLs with `#t=0.1` thumbnail preview.
Android Client	`/android/.../OpenShortsViewModel.kt` `/android/.../FluxClipScreen.kt`	Add SAF pickers for `.srt/.txt` and `.mp3/.wav`, `ContentResolver` metadata/duration extraction, chunked upload calls, and validation UI.

PART 4: RECOMMENDED IMPLEMENTATION ORDER

1. **Step 1:** Update shared contracts and pure validation/mapping logic in `/src/utils/pipelineLogic.ts` and `/src/utils/pipelineLogic.test.ts`.
2. **Step 2:** Upgrade `/server.ts` with offset-verified chunked uploads, `/usr/bin/ffprobe` validation, real `/usr/bin/ffmpeg` child-process slicing & subtitle burn-in, and binary `video/mp4` delivery.
3. **Step 3:** Upgrade `/src/viewmodels/useOpenShortsViewModel.ts`, `/src/components/UploadsScreen.tsx`, and `/src/components/PodcasterFrame.tsx`.
4. **Step 4:** Upgrade Android `OpenShortsViewModel.kt` and `FluxClipScreen.kt`.
5. **Step 5:** Execute unit tests, TypeScript build & lint checks, and the 20-scenario end-to-end verification matrix.

PART 5: 20-SCENARIO ACCEPTANCE TEST MATRIX

#	Scenario	Expected Backend Operation	Expected Frontend Response	Pass/Fail Criteria
1	Small local MP4 (<512 KB)	`/api/upload/init` → `chunk` → `complete` verifies size & `ffprobe` duration.	Displays filename, size, and duration.	`receivedBytes === totalSizeBytes` & valid duration.
2	Large local MP4 (>512 KB)	Sequential chunk writes at exact `byteOffset` .	Shows byte upload percentage.	Staged file matches original byte-for-byte.
3	Local video + SRT + MP3	`ffprobe` sync check, `ffmpeg` slice & `.ass` burn-in.	Transitions to Screen 2 with playable 9:16 clips.	Output `.mp4` files exist on disk (`size > 0`).
4	Local video + TXT + WAV	Parses `.txt` cues, slices `.wav` , muxes AAC into MP4.	Transitions to Screen 2 with rendered clips.	Zero FFmpeg muxing errors.
5	Direct MP4 URL + SRT + MP3	SSRF check, remote `ffprobe` duration check, range-based `ffmpeg` slice.	Streams rendered clip MP4 on Screen 2.	Remote video duration validated & clip plays.
6	Supported YouTube URL	Resolves stream or returns structured restriction error.	Loads clips or displays clear error banner.	Never passes fake `yt-dlp://` scheme to FFmpeg.
7	Invalid YouTube URL	Rejects with HTTP 400 (`INVALID_URL`).	Shows inline validation error.	Blocked before job creation.
8	Expired remote video URL	`ffprobe` fails; returns HTTP 422 (`REMOTE_SOURCE_UNAVAILABLE`).	Displays error; resets `isGenerating` .	Halts cleanly without hanging.
9	Missing master audio	Rejects with HTTP 422 (`MISSING_MASTER_AUDIO`).	Prompts user to upload master audio.	No job created without audio binary.
10	Missing transcript	Rejects with HTTP 422 (`MISSING_TRANSCRIPT`).	Prompts user to upload transcript.	No job created when transcript is empty.
11	A/V duration mismatch (>2.5s)	`ffprobe` compares true video vs audio duration; rejects with HTTP 422.	Shows exact duration mismatch message.	Rejects drift > 2500ms on local & remote inputs.
12	Invalid transcript timestamps	Rejects malformed or out-of-bounds cues with HTTP 422.	Shows cue-specific timestamp error.	Out-of-bounds cues blocked.
13	Gemini API failure	Falls back to validated transcript cues or fails cleanly.	Completes with cue hooks or shows error.	Server never crashes; reaches terminal state.
14	FFmpeg execution failure	Non-zero exit kills process, cleans staging, sets `failed` .	Shows error and resets button state.	Failed FFmpeg never marks job `completed` .
15	Faster-Whisper failure	Empty/invalid word stream transitions job to `failed` .	Shows alignment error and resets button.	Invalid word timestamps rejected.
16	Processing > 3.6 seconds	Transitions through granular stages until completion.	Polls every 1s until terminal state.	Jobs taking 5–60s complete cleanly.
17	Multiple clips in carousel	Generates distinct `clip-1.mp4` , `clip-2.mp4` , etc.	Carousel switches `<video src>` per clip.	Each clip plays its own unique time slice.

#	Scenario	Expected Backend Operation	Expected Frontend Response	Pass/Fail Criteria
18	Actual MP4 browser playback	`GET /api/artifacts/...` serves `video/mp4` with HTTP 206 Range.	`<video>` plays, pauses, and scrubs.	`readyState >= 2` and zero console errors.
19	Actual MP4 download	Serves binary MP4 with `Content-Disposition: attachment`.	Downloads valid playable `.mp4` file.	Starts with MP4 `ftyp` signature, never JSON.
20	Android file pickers	SAF pickers select video, `.srt/.txt`, and `.mp3/.wav`.	Displays filenames, sizes, and remove actions.	All 3 inputs selectable and validated in Compose.